Project Update 3: Requirements, Design, and Early Implementation

Team: A. Role: Developer 2 (QA & Testing)
Date:10/15/2026

1. Current Objective

The team is currently developing the "Personal Finance Manager" (PFM) application using Agile practices. For this sprint, our primary objective is to deliver a small, useful increment: Enhancing usability by adding the ability to Delete existing budget items.

- User-Visible Change: The user can select existing transactions and delete those with incorrect information.

- Smallest Useful Slice: The user can select and delete existing transactions, which are then immediately removed from the GUI table and the underlying expenses.json file.

- Acceptance Criteria & Definition of Done: The feature is considered "done" when a selected transaction is removed, the updated list saves correctly to the JSON file, the remaining transactions load correctly upon reopening the app, and a second team member (QA) can verify the tested change.

In addition to this core feature, we are refining the UI formatting and addressing edge cases (like empty table interactions). Our initial tasks focused on identifying key dependencies and risks—specifically, verifying that the delete action saves back to the JSON file correctly upon close/reopen, and ensuring the correct data is targeted for deletion without crashing the app.

2. Your Contribution

As Developer 2 (QA/Tester), I personally executed the testing and quality assurance workflow for the `dev` branch. My specific contributions include:

- **Baseline Evaluation** (**9/19**): Captured the initial state and core functionality of the app prior to the UI formatting updates, establishing a baseline for future regression testing.
- **Edge Case & Persistence Testing** (**9/23**): Tested the "Delete Expense" feature to ensure it handles empty selections gracefully and correctly updates the `expenses.json` file.
- **Regression Testing** (**9/24**): Re-tested the delete functionality and UI popups after Dev 1 pushed new UI formatting to ensure no existing features were broken.
- **Empty Table Testing** (**9/28**): Executed tests for Task #14 to verify how the "Show Summary" and "Visualize Spending" buttons behave when the data table is completely empty.

- **Bug Verification & Final Testing (9/30)**: Pulled the latest dev branch to verify the Lead Developer's fixes for the "Disabled Delete Button" and the "Visualize Spending" crash bug.
- **Edit Workflow & Edge Case QA** (**10/14**): Executed comprehensive QA testing for Issue #17 (Edit Expense pop-up). Verified happy paths (triggers, save/persistence, cancel) and edge cases (blank field validation). Identified and documented a minor UI bug regarding button state on zero-selection, which the Lead Dev acknowledged and committed to fixing.
- **Code Review & Refactor Verification (10/15)**: Reviewed the code diff and commit history for Issue #20 (QoL auto-increment refactor). Verified the UI updates correctly and performed regression testing to ensure existing data loads and saves without issues.
- **Process Organization & Evidence Tracking**: Established and continuously updated a structured evidence repository **(ProjectUpdate2_Dev2_Evidence-Rev3)** to systematically track screenshots, GitHub logs, and test documentation. This repository now contains 33 figures, including the latest QA sign-offs and bug reports for Issues #17 and #20.

3. Evidence

All observable evidence supporting this update is organized in the **ProjectUpdate2_Dev2_Evidence-Rev3** folder and includes:

- GitHub Issues **#10 & #14,#17, & #20**: Contains timestamped comments documenting Pass/Fail results, the formal bug report for the Task #14 crash, and the Lead Developer's documentation of the fix.
- GitHub Projects Board: Shows issues claimed, assigned, moved through the workflow, and officially moved to "Done" after rigorous QA validation (with Issue #17 pending a minor, agreed-upon polish).
- Application Screenshots (**Figures 1–33**): Visual proof of the baseline app state, UI formatting, yellow warning popups, resolved crash states, and the newly implemented Edit pop-up workflow and validation checks.
- **Test Execution Log**: A detailed breakdown of Expected vs. Actual results for all tested features (see Appendix).
- **Meeting Notes & Team Communication**: Screenshots from team meetings documenting our agreed-upon Git workflow, collaborative decisions on edge case testing, and real-time status updates between the Lead Developer and Dev 2.

4. Issue or Risk

- Resolved Defects: Fixed the "Visualize Spending" crash and completed the "Disabled Delete Button" logic. Both now correctly disable when no items are selected.
- Scope Risk Mitigation: Deliberately deferred Issue #19 to prevent scope creep, and evaluating descoping Issue #1's "remove" feature to protect delivery timelines.
- QA Environment Protection: Consistently used git restore expenses.json after local testing to prevent accidental overwrites of the team's shared dat

5. Next Step

With QA validation complete for Issues #17 and #20, my immediate testing objectives for this cycle are met. Before the next update, I will:

- Monitor & Close: Track the minor UI polish for Issue #17, ensuring it meets our Definition of Done before formally moving it to "Done" on the GitHub board.
- Backlog Refinement: Collaborate with the team to triage upcoming work (e.g., UI improvements, date formatting, and the deferred Issue #19), drafting formal Acceptance Criteria checklists to enable a proactive "shift-left" testing approach.
- Finalize & Prepare: Submit this Project Update 3 documentation and prepare my local testing environment (utilizing git restore protocols) for the next batch of user stories.

Appendix: Test Execution Log & Visual Evidence

Test Case 1: Data Persistence & Edge Cases (9/23)

Action: Selected "Electric Bill" (a middle row) and clicked "Delete Expense(s)". Closed and reopened the app to verify persistence.

- Result: PASSED. The row disappeared from the table immediately. Upon reopening, "Electric Bill" remained deleted, confirming it was successfully removed from the expenses.json file.

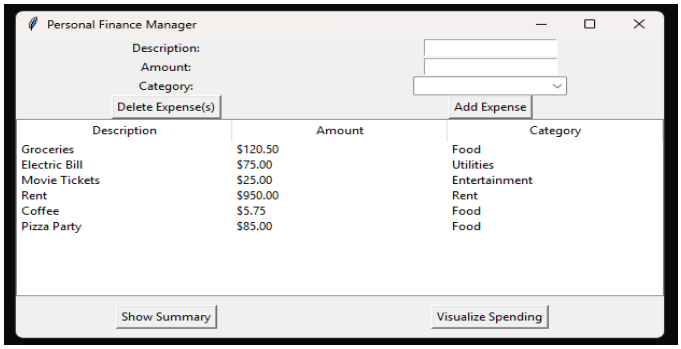


Fig1: Initial state of the app with all data present.

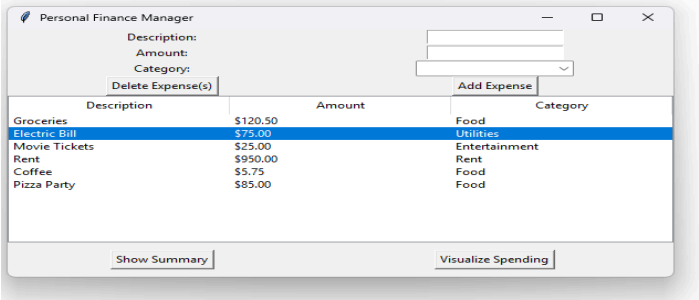


Fig2:(Selecting a middle row to test deletion.)

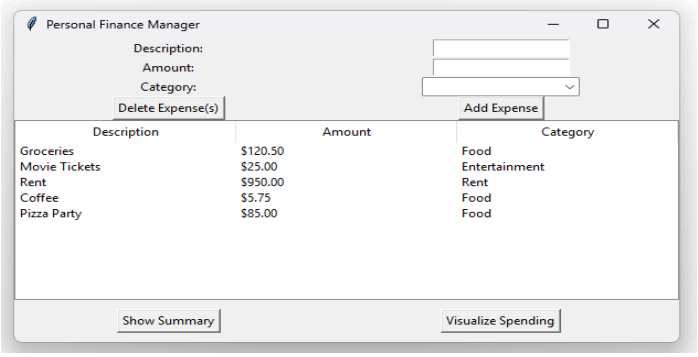


Fig3:The row is removed from the UI immediately.

Test Case 2: GitHub Board & Documentation (9/23)

Action: Managed Agile workflow and documented results on GitHub.

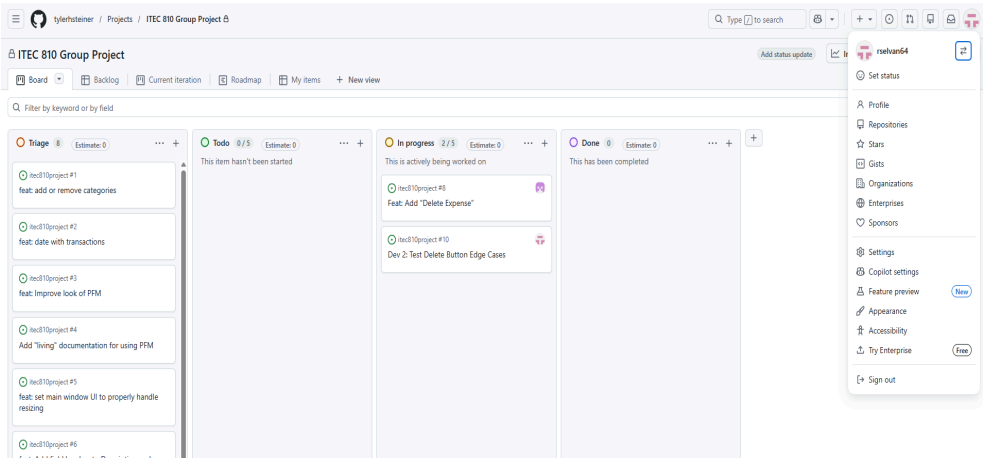


Fig4_GitHub_Projects_Board: GitHub Projects Board showing Issue #10 in the "In Progress" column.

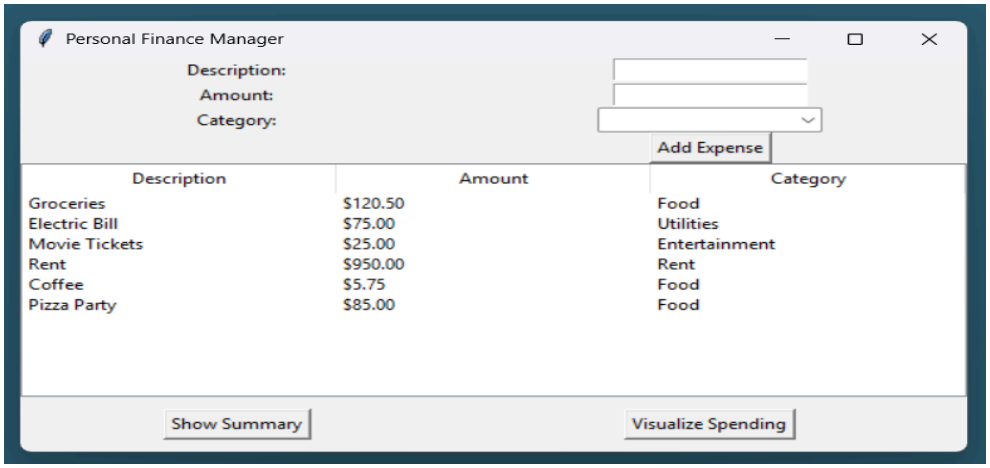


Fig 5: Baseline app state on 9/19 prior to UI formatting updates

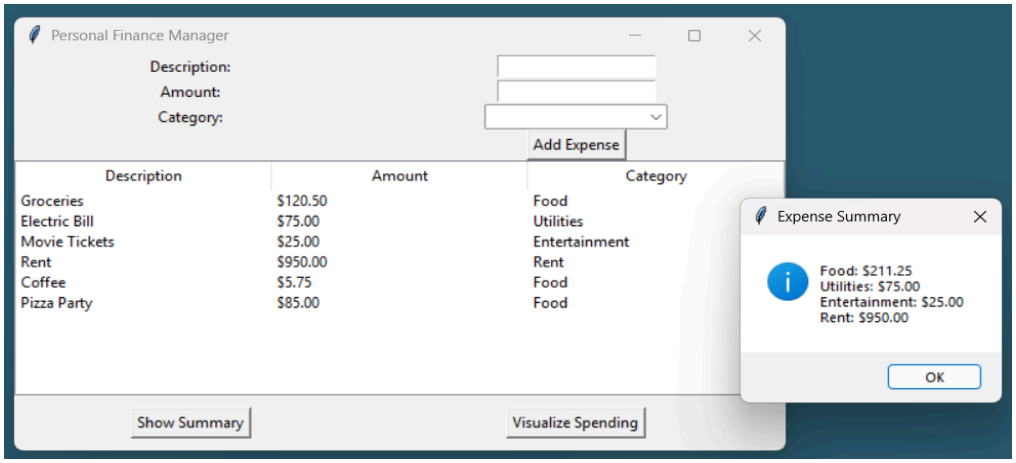


Fig 6: Baseline "Expense Summary" popup functionality on 9/19, confirming the feature worked before the UI overhaul.

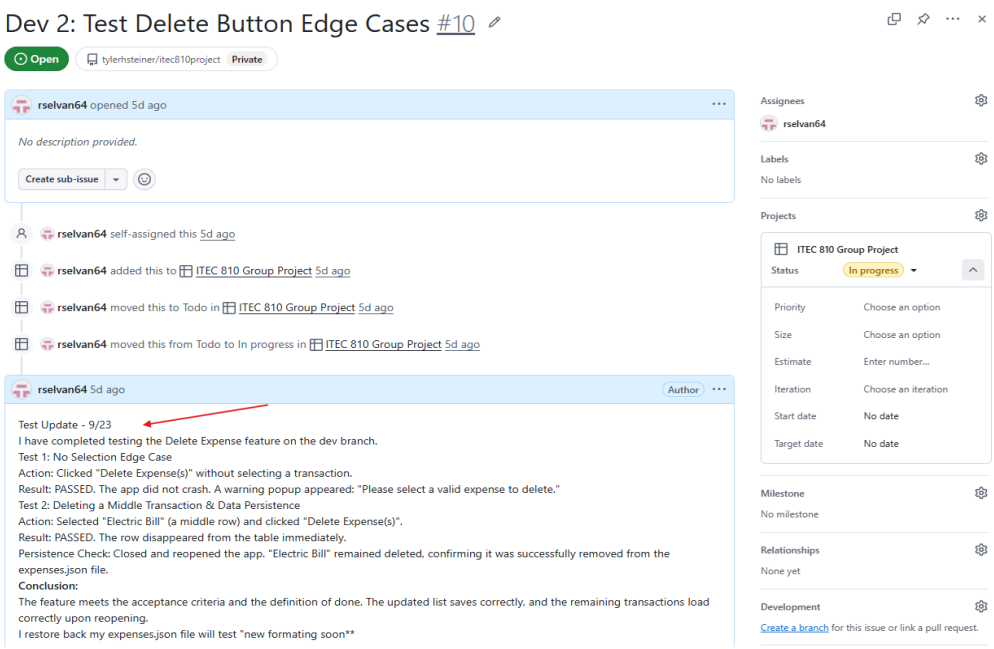


Fig7_GitHub_Board:Red Arrow shows evidence. Updated board status showing active task management.

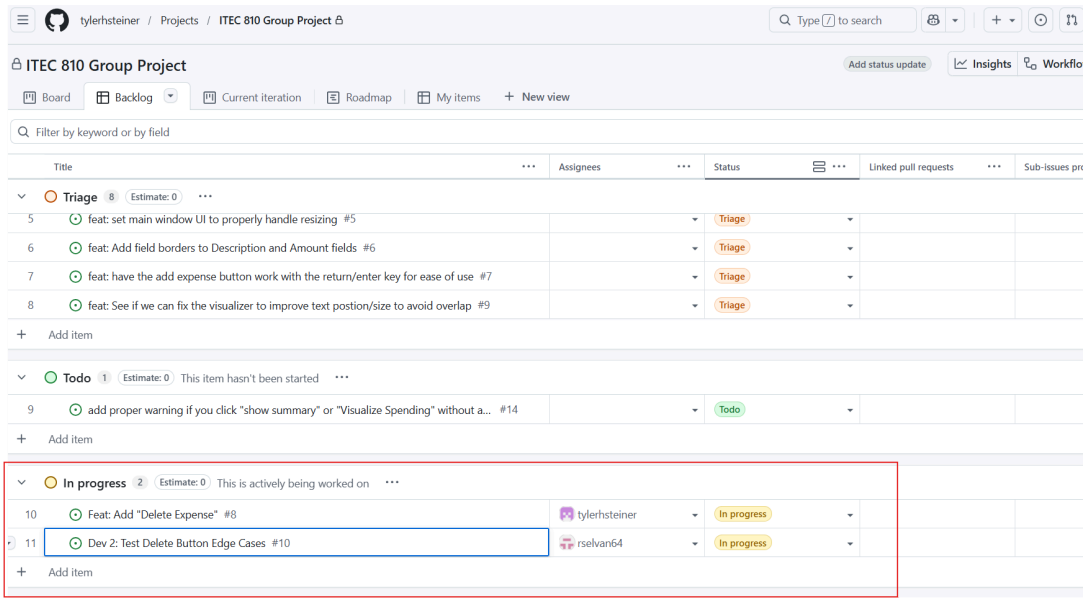


Fig8_GitHub_Issue#10:Detailed timestamped test results and assignee status on Issue #10.

Test Case 3: Regression Test Post-UI Update (9/24)

Action: Re-tested edge cases and features after Dev 1 pushed new UI formatting.

- Result: PASSED. The new UI formatting did not break existing functionality. The yellow warning popup and data persistence continued to work perfectly.

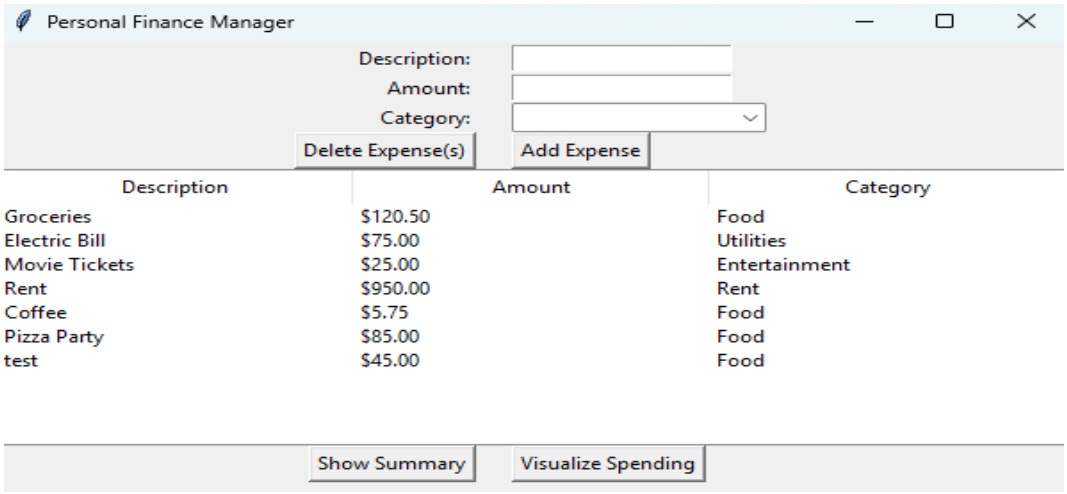
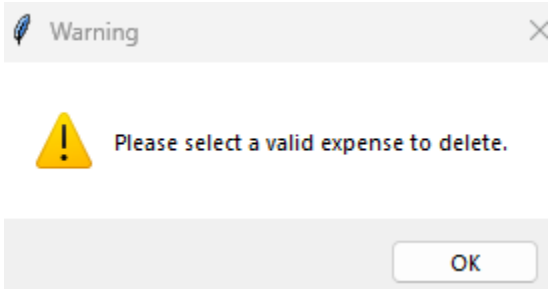


Fig9:.App state immediately after pulling the new UI code on.



(Fig.10: Result: Edge case warning still functions correctly after UI updates.)

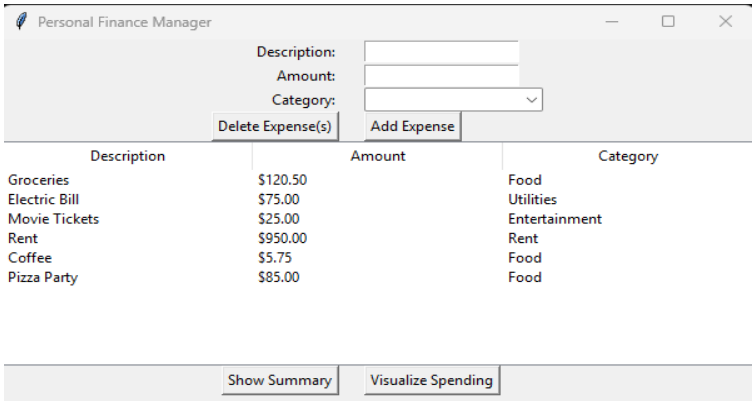


Fig11: Result: Data persistence still works perfectly on the new UI.

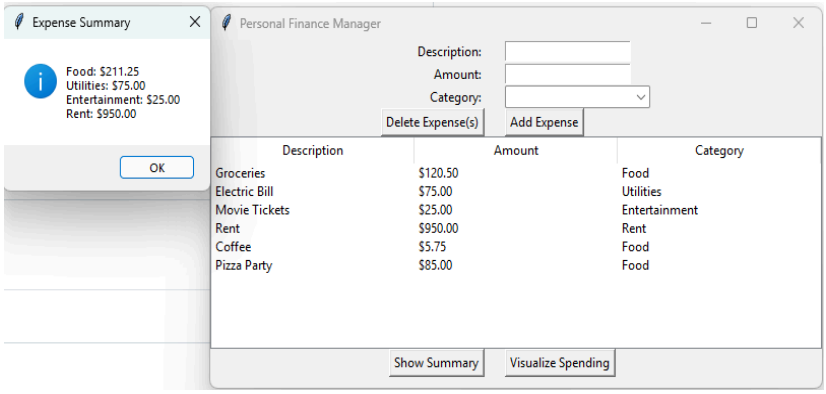


Fig12: Result: Show “Summary"popup renders correctly with the new UI.

Test Case 4 & 5: Empty Table Edge Cases (9/28)

Action: Cleared all rows from the table to test Task #14 ("Show Summary" and "Visualize Spending").

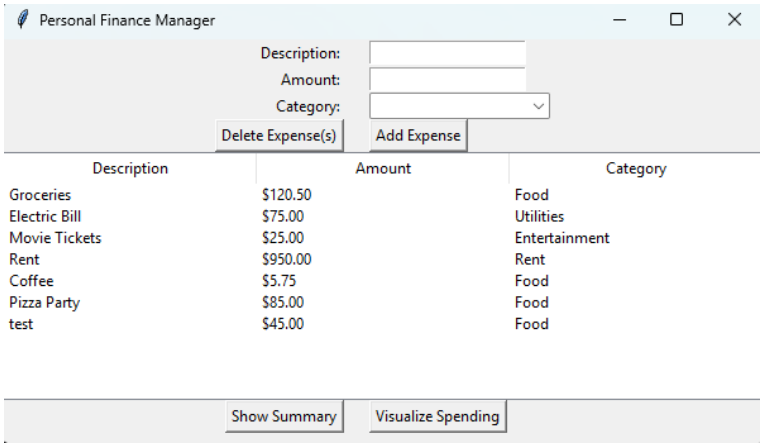


Fig13:App fully loaded with data before clearing the table.

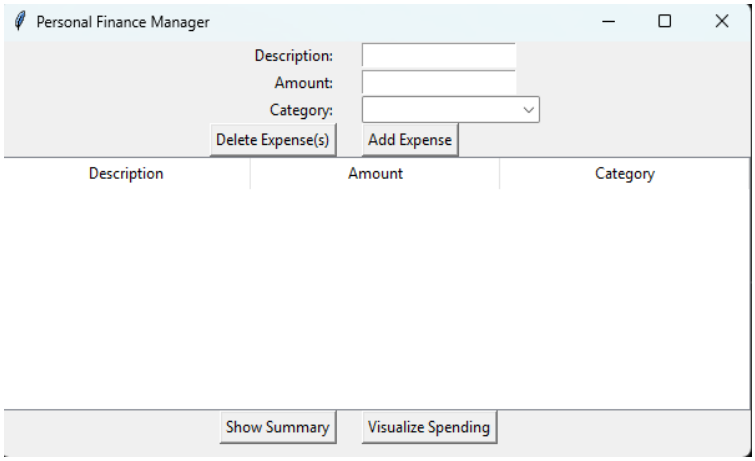


Fig14:Table cleared of all data to test edge cases.

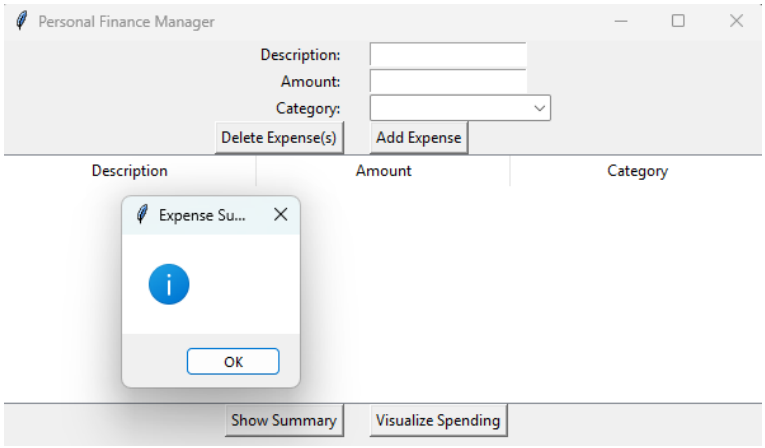


Fig15: Result (Test 4): PASSED. The app did not crash. It displayed an empty summary popup gracefully.

```
Exception in Tkinter callback
Traceback (most recent call last):
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\tkinter\__init__.py", line 2114, in __call__
    return self.func(*args)
           ~~~~~~~~~^^^^^^^
  File "C:\Users\rselv\OneDrive\Desktop\itec810project\main.py", line 158, in visualize_spending
    plt.pie(totals, labels=categories, autopct='%1.1f%%')
    ~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\_api\deprecation.py", line 477, in
wrapper
    return func(*args, **kwargs)
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\pyplot.py", line 3989, in pie
    return gca().pie(
           ~~~~~~~~~^
        x,
        ^^
    ...<17 lines>...
        **({"data": data} if data is not None else {}),
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\_api\deprecation.py", line 477, in
wrapper
    return func(*args, **kwargs)
```

```
    return func(*args, **kwargs)
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\pyplot.py", line 3989, in pie
    return gca().pie(
           ~~~~~~~~~^
        x,
        ^^
    ...<17 lines>...
        **({"data": data} if data is not None else {}),
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\_api\deprecation.py", line 477, in
wrapper
    return func(*args, **kwargs)
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\__init__.py", line 1531, in inner
    return func(
        ax,
        *map(cbook.sanitize_sequence, args),
        **{k: cbook.sanitize_sequence(v) for k, v in kwargs.items()})
  File "C:\Users\rselv\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\matplotlib\axes\_axes.py", line 3667, in pie
    raise ValueError('All wedge sizes are zero')
ValueError: All wedge sizes are zero
```

Fig16:VisualizeSpending(Test 5): FAILED. The app crashed with a **ValueError: All wedge sizes are zero** because it tried to draw a pie chart with empty data. Reported to Dev 1 board for a fix.)

Test Case 6: Team Collaboration & Meeting Notes

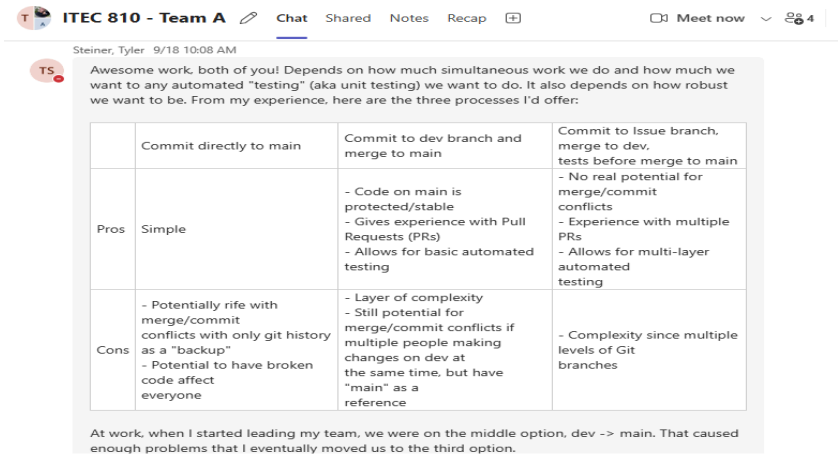


Fig17:Team1_Scope_Git_Workflow: Team Meeting establishing the Git Branching Strategy,scope,and merge protocols)

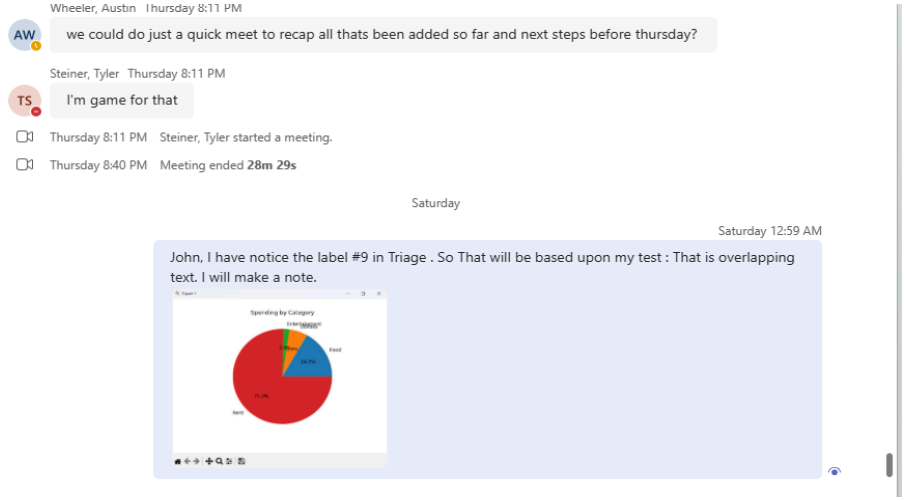


Fig18: Team2_Edge_Cases_Issue: Collaborative discussion on edge cases. Identified overlapping text in the Visualize Spending pie chart (Issue #9) for future testing.)

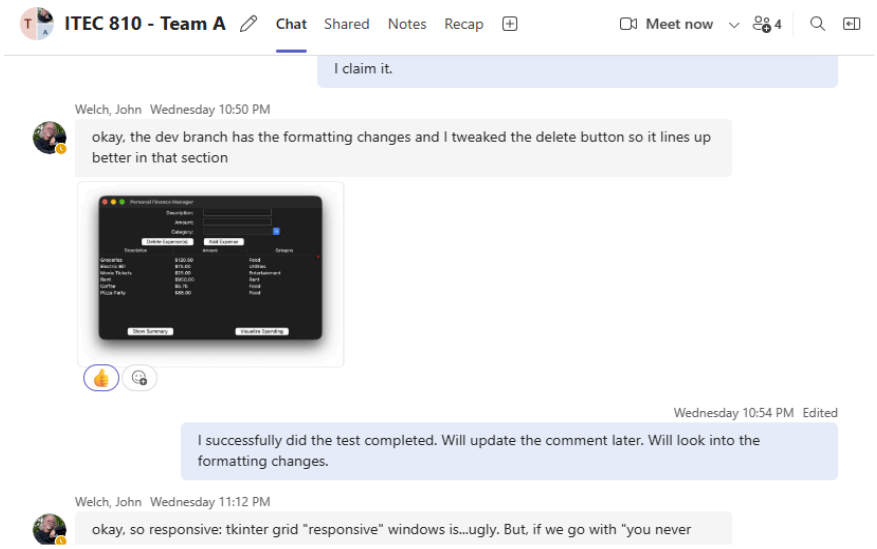


Fig19:Team3_UI_Updates_Testing_Confirmation: Dev 1 confirmed UI formatting pushes to the dev branch. Dev 2 confirmed successful completion of regression tests.)

Test Cases 7: VErification of Lead Developer Fixes (9/30)

Action: Pulled the fresh `dev` branch to test the Lead Developer's updates regarding the disabled buttons and the empty table crash.

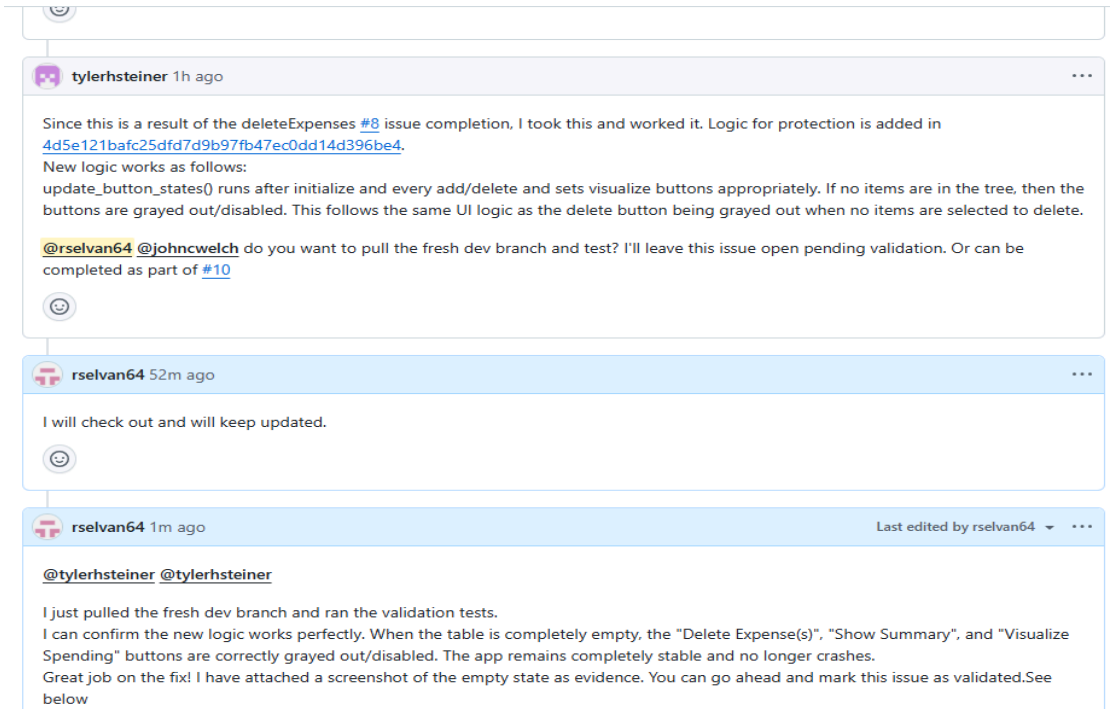


Fig 20: Lead Developer's GitHub comment detailing the new logic to disable buttons when the tree is empty.)

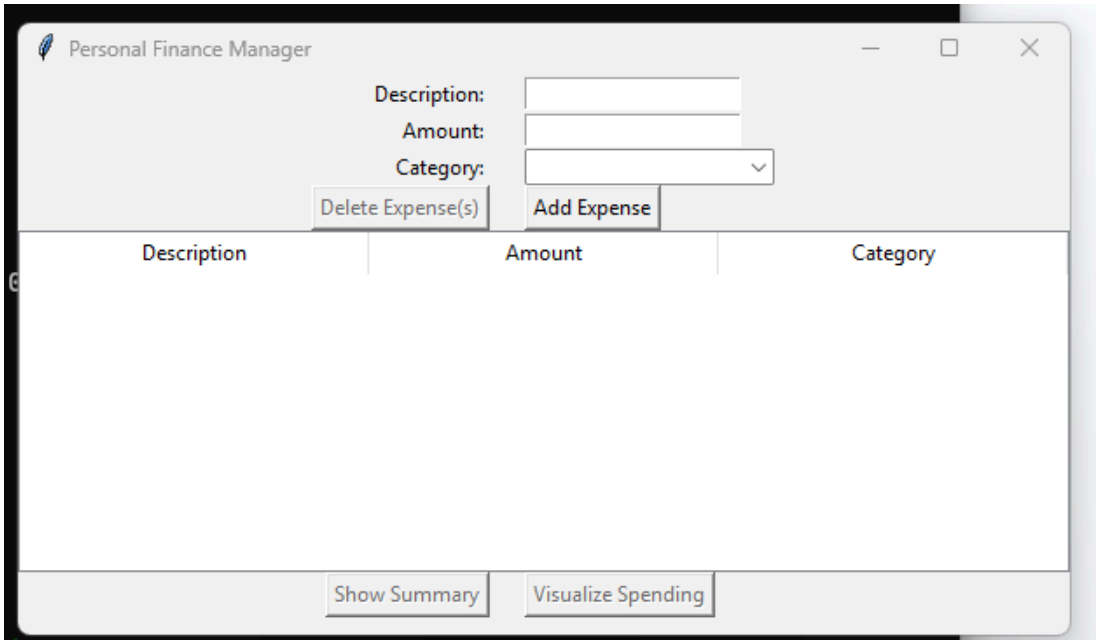


Fig 21: Result (Test 7: PASSED. The buttons are now correctly grayed out/disabled when no data is present, preventing the previous crash.)

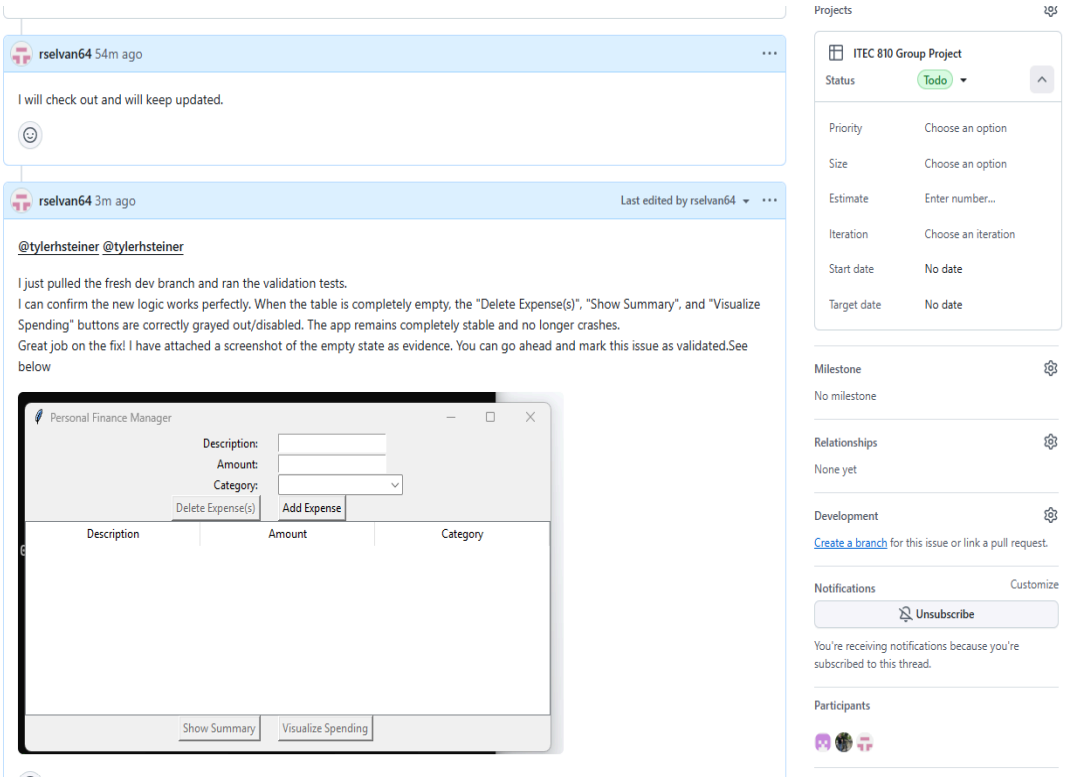


Fig 22: Final Outcome: After Dev.2(QA) Validation, the Lead Developer confirmed the fix and moved the issue to the "Done" column on the GitHub Projects board, officially closing the loop.)

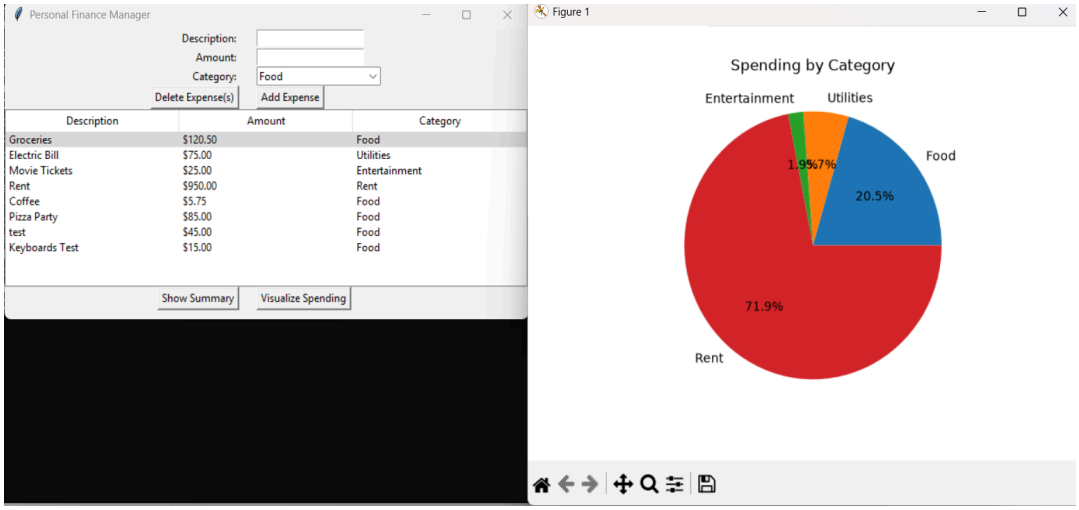
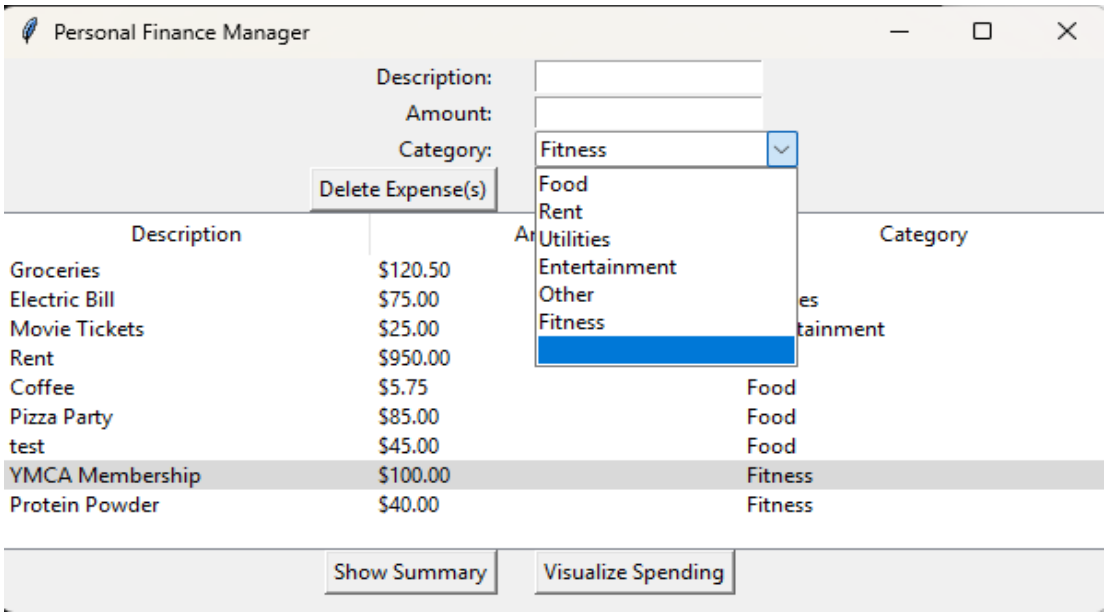


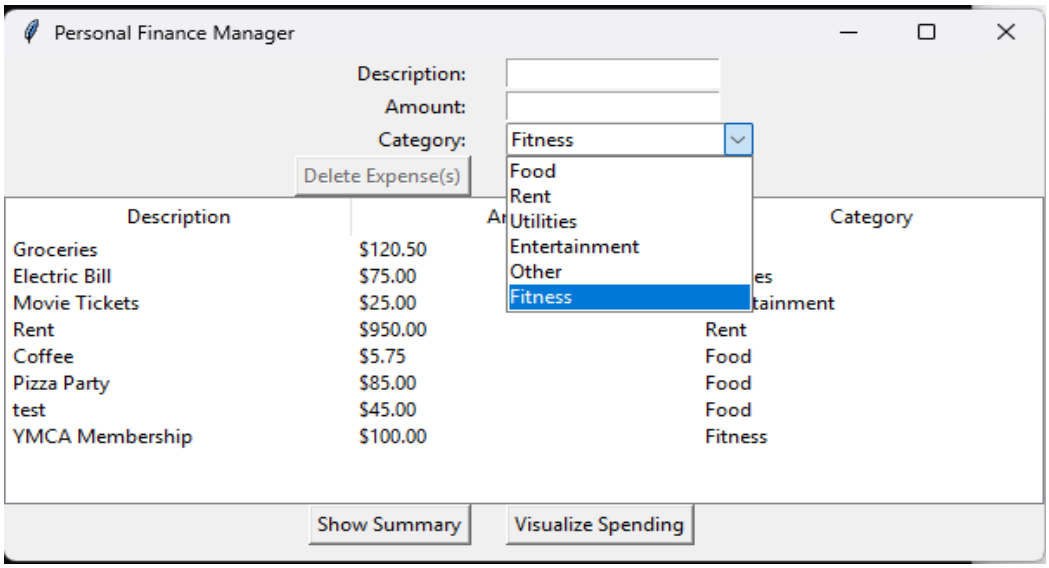
Fig23:Visualize_Spending_Verification: **GitHub #6**: Visualize Spending Verification (10/3). The pie chart renders correctly and updates the "Food" category after adding the test expense via the Enter key, confirming no regression in existing functionality.

Test Case: 10/4: GitHub #1: Happy Path (Auto-Adding a New Category : Passed. Added new brand new category: YMCA Membership expense is added to the table.Dropdown appears.



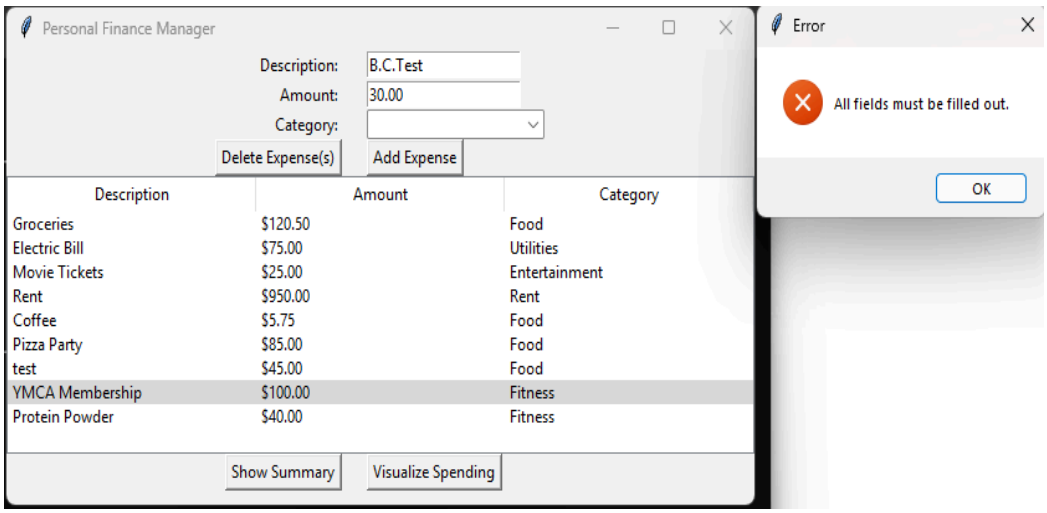
(Fig24_New_Category_Added): Github #1: Auto-Adding a New Category

Test Case: **Data Persistence** (JSON File). Close it and restart and no crash.



(Fig25_**Category_Persistence**) Github #1:Passed:The app opens normally. When you click the dropdown, "Fitness" is still there

Test Case: **Edge Case** (Empty Category)
The app did not crash. Shows a validation warning popup and rejects the entry. It should definitely not add a blank/empty space to the category dropdown list.



(Fig26_Empty_Category_Warning) Github #1: Passed: If a warning popup appears !

Oct.15th Update.
Evidence for Issue #17 (Edit Expense Workflow)

Test Case: Happy Path (Edit Triggers)

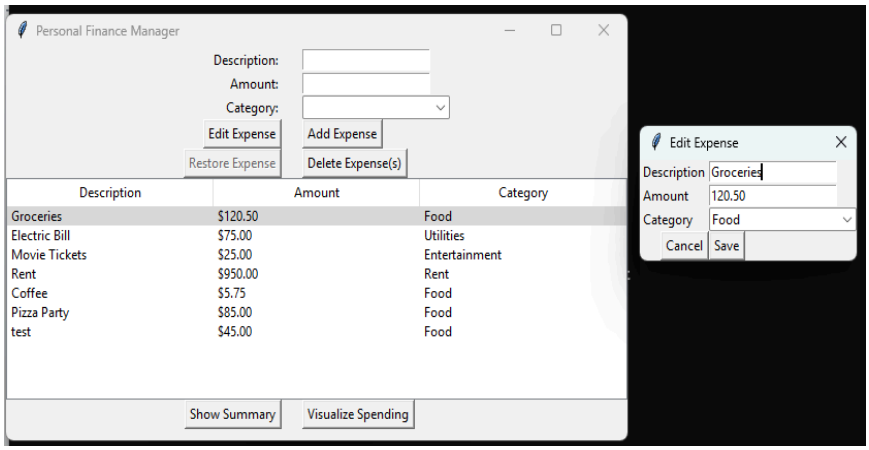


Figure 27: Trigger_Pass
GitHub #17: Passed: Verified that both double-clicking a row and using the "Edit Expense" button successfully trigger the pre-filled pop-up dialog.

Test Case: Happy Path (Save Action & Persistence)

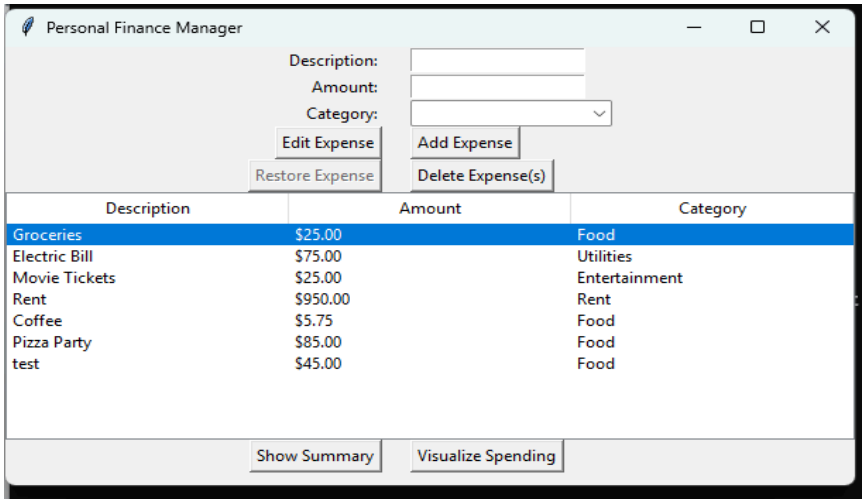


Figure 28: Save_Amount_Update.
GitHub #17: Passed: Edited amount to $25.00, clicked Save, and confirmed the main UI updated immediately and persisted to expenses.json.

Test Case: Edge Case (UI State / Selection Logic)

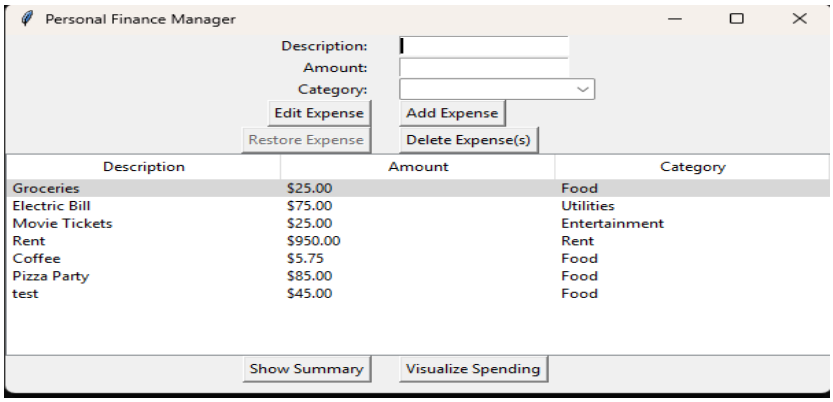


Figure 29: Zero_Selection_Bug

GitHub #17: Bug Found: Identified that the "Edit Expense" button remained active/clickable when 0 rows were selected, violating the requirement that it should only be enabled for exactly one selection.

Test Case: Edge Case (Input Validation)

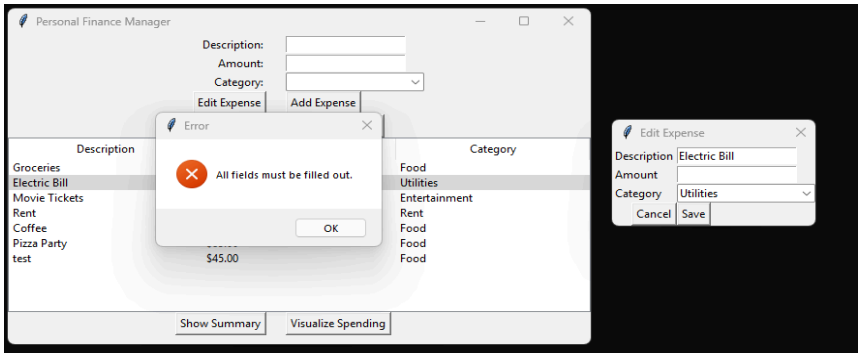


Figure 30: Blank_Field_Validation

GitHub #17: Passed: Attempted to save with a blank "Amount" field; the system correctly prevented the save and displayed the "All fields must be filled out" warning popup without crashing.

Test Case: Documentation & Team Communication

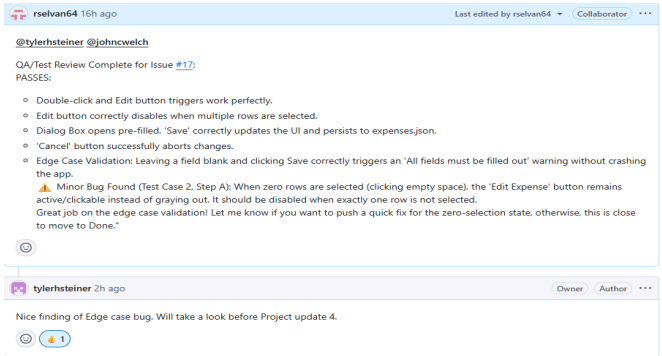


Figure 31: GitHub_QA_SignOff_Comment

GitHub #17: Passed: Formal QA sign-off comment posted documenting the passes and the minor bug. Lead Dev acknowledged ("Nice finding of Edge case bug") and committed to patch before Update 4.

Test Case: Code Review & Understanding

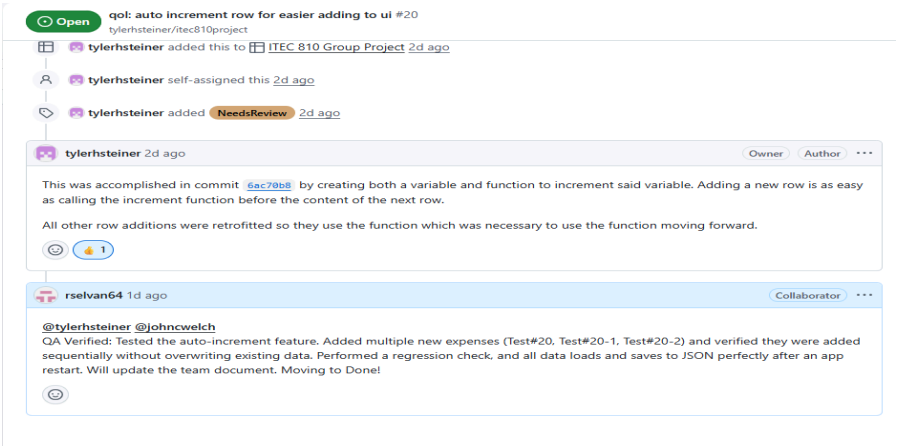


Figure 32: AutoIncrement_Code_Diff .

GitHub #20: Passed: Reviewed the developer's code diff and commit (6ac70b8) to verify the refactoring of hardcoded rows into a dynamic ui_row variable

Test Case: Regression Testing Verification

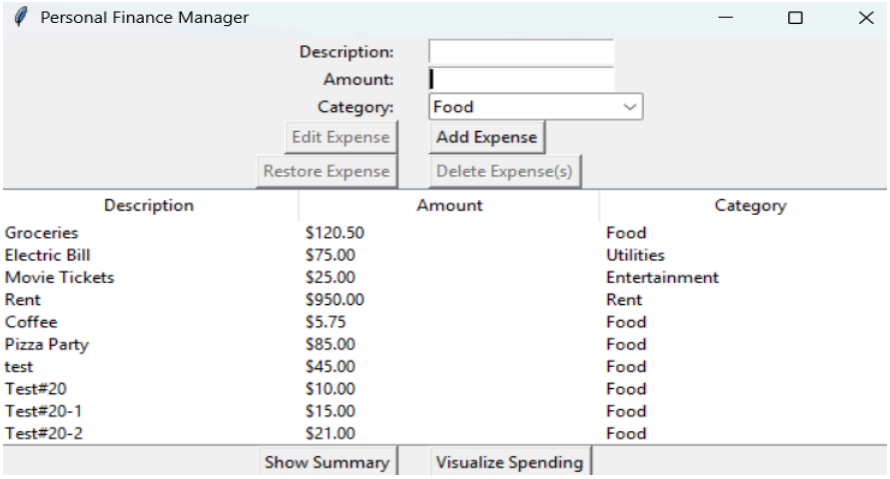


Figure 33:GitHub_QA_SignOff_Complete

GitHub #20: Passed: Formal QA verification posted confirming the auto-increment feature works correctly and regression testing on existing rows passed without issues.